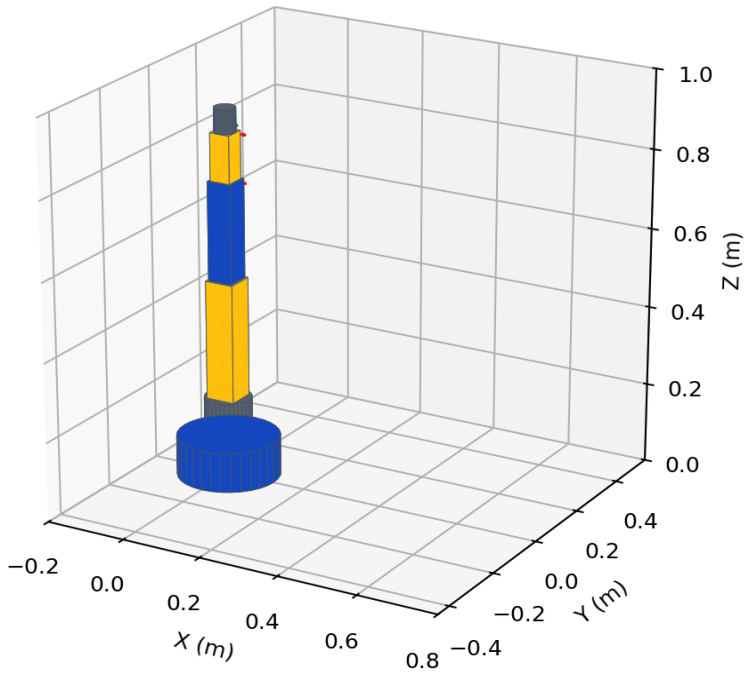
ROS 2 URDF of a 4-DOF Industrial Arm

URDF Modeling and Kinematic Validation

Mini Project Report

Home: q1=q2=q3=q4=0
URDF-derived render — not RViz



Student: Deep Bisen

Date: 23 October 2026

Abstract: A serial 4-DOF robot arm is defined in URDF with visual, collision and inertial elements, bounded revolute joints and a tool frame. An ament_cmake package supplies ROS 2 launch and RViz configuration. The URDF was parsed and its geometry, tree and forward kinematics checked. Actual ROS 2 execution, RViz visualization and live TF screenshots remain pending because ROS 2 was unavailable here.

1. Robot structure and kinematics (Q1)

The arm is a serial yaw-pitch-pitch-pitch mechanism. Joint 1 rotates the shoulder assembly about the vertical Z axis. Joints 2 and 3 move the upper arm and forearm about their local Y axes. Joint 4 pitches the wrist. Fixed joints attach the base to world and the tool to the wrist. Four independent revolute coordinates give four actuated degrees of freedom; the fixed joints add none.

Link / frame	Geometry and length	Mass
world	Empty reference frame	None
base_link	Cylinder radius 0.12 m, height 0.10 m	3.0 kg
shoulder_link	Cylinder radius 0.055 m, height 0.10 m	1.0 kg
upper_arm_link	Box 0.07 x 0.07 x 0.30 m	2.0 kg
forearm_link	Box 0.06 x 0.06 x 0.25 m	1.5 kg
wrist_link	Box 0.05 x 0.05 x 0.12 m	0.5 kg
tool_link	Cylinder radius 0.025 m, height 0.06 m	0.2 kg

SI units are used: metres, radians, kilograms, N m and rad/s. Link frames lie at their proximal joint. All home link lengths extend along local +Z. Visual/collision geometry and inertial centres are translated by half the link length, so joint frames meet at segment ends. The root world frame is a reference only; all six physical links have geometry and inertia.

This mechanism cannot realize an arbitrary six-dimensional tool pose. The wrist has pitch only, with no independent tool roll. It is an educational industrial-arm representation rather than a manufacturer-specific robot or a complete controlled manipulator.

2. URDF link definitions (Q2)

Each physical link contains visual and collision geometry. Primitive cylinders/boxes avoid missing mesh paths and keep the model portable. The same nominal shape is used for collision; visual colors distinguish blue structure, yellow moving segments and grey interfaces. Collision elements define shapes for downstream tools but do not themselves detect or prevent self-collisions in RViz.

Every physical link has positive mass and a centre-of-mass inertial origin. A uniform-density primitive approximation is used. For a box, $I_{xx}=m(b^2+c^2)/12$, $I_{yy}=m(a^2+c^2)/12$ and $I_{zz}=m(a^2+b^2)/12$. For a solid cylinder along Z, $I_{xx}=I_{yy}=m(3r^2+h^2)/12$ and $I_{zz}=mr^2/2$. Products of inertia are zero in these principal-axis frames. These are assumed model properties, not measured actuator/link data.

Representative upper-arm definition (complete implementation is in the URDF file):

```
<link name="upper_arm_link">
  <visual>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
    <geometry><box size="0.07 0.07 0.30"/></geometry>
    <material name="yellow"/>
  </visual>
  <collision>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
    <geometry><box size="0.07 0.07 0.30"/></geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
    <mass value="2.0"/>
    <inertia ixx="0.015816666667" ixy="0" ixz="0"
             iyy="0.015816666667" iyz="0"
             izz="0.001633333333"/>
  </inertial>
</link>
```

The source URDF retains full numeric precision. All six inertia matrices were checked for positive eigenvalues and physical principal-moment triangle inequalities. No claim is made about realistic payload, stiffness, controller stability or actuator sizing.

3. Joint definitions and limits (Q3)

Joint	Parent -> child / origin Z	Axis / limits
world_fixed	world -> base_link; 0 m	Fixed
joint1_base_yaw	base_link -> shoulder_link; 0.10 m	Z; -pi to +pi
joint2_shoulder	shoulder_link -> upper_arm_link; 0.10 m	Y; -pi/2 to +pi/2
joint3_elbow	upper_arm_link -> forearm_link; 0.30 m	Y; -2pi/3 to +2pi/3
joint4_wrist	forearm_link -> wrist_link; 0.25 m	Y; -pi/2 to +pi/2
tool_fixed	wrist_link -> tool_link; 0.12 m	Fixed

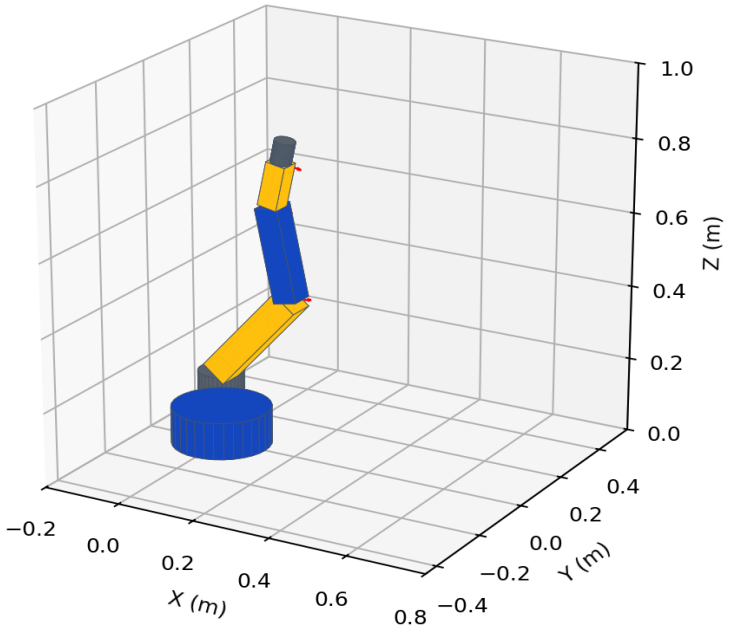
Joint origins are expressed in the parent-link frame; all origin RPY values are zero. Joint axes are expressed in the joint frame. Positive rotation follows the right-hand rule. Effort limits for joints 1–4 are 50, 40, 25 and 10 N m; velocity limits are 1.0, 1.0, 1.2 and 1.5 rad/s. Damping=0.1 and friction=0.05 are illustrative URDF parameters, not identified physical values.

```
<joint name="joint3_elbow" type="revolute">
  <parent link="upper_arm_link"/>
  <child link="forearm_link"/>
  <origin xyz="0 0 0.3" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit lower="-2.0943951023931953"
         upper="2.0943951023931953"
         effort="25" velocity="1.2"/>
  <dynamics damping="0.1" friction="0.05"/>
</joint>
```

Revolute joints are chosen instead of continuous joints because the educational arm has bounded travel. These limits constrain the description and joint-state GUI range; they do not constitute a safety-rated stop or hardware controller. The described ranges have not been certified collision-free.

4. Assembly and kinematic validation (Q4)

Bent pose: q=[0, 0.7, -0.9, 0.4] rad
URDF-derived render — not RViz



A single URDF joins all seven frames with six joints. Each non-root link has exactly one parent. The parser identifies world as the sole root. Tree traversal reaches every link, rejecting loops or disconnected branches. Visual/collision shapes and positive mass/inertia were verified for every physical link. The supplied figures are generated from the actual URDF geometry and joint transforms, not RViz screenshots.

Forward kinematics composes parent transforms with joint-origin translation and axis rotation: Tchild = Tparent Torigin Raxis(q). Since all origin RPY values are zero in this specific model, the validation script uses translation plus Rodrigues rotation. It is intentionally not a general-purpose URDF dynamics engine.

Test angles (q1,q2,q3,q4)	Expected tool-frame XYZ (m)
(0, 0, 0, 0)	(0, 0, 0.87)
(0, pi/2, 0, 0)	(0.67, 0, 0.20)
(0, 0, pi/2, 0)	(0.37, 0, 0.50)
(pi/2, pi/2, 0, 0)	(0, 0.67, 0.20)

All four analytic target poses matched within 1e-10 m in the local checks. A shoulder angle of 2 rad was rejected as out of bounds. The home tool frame lies at Z=0.87 m; the distal face of the 0.06 m tool lies at Z=0.93 m. Frame position must not be confused with the outer tip position.

5. ROS 2 launch and RViz procedure (Q5)

Target workflow: ROS 2 Humble on Ubuntu 22.04 with a graphical desktop. Install robot_state_publisher, joint_state_publisher_gui, RViz2, tf2 tools and colcon using official ROS instructions. Full commands are in README.md. The package also uses launch_ros and ament_index_python. Iron is permitted by the assignment but was not tested.

```
source /opt/ros/humble/setup.bash
mkdir -p ~/ros2_ws/src
# Copy the included arm_description folder into src/
cd ~/ros2_ws
colcon build --symlink-install --packages-select arm_description
source install/setup.bash
check_urdf src/arm_description/urdf/industrial_arm.urdf
ros2 launch arm_description display.launch.py
```

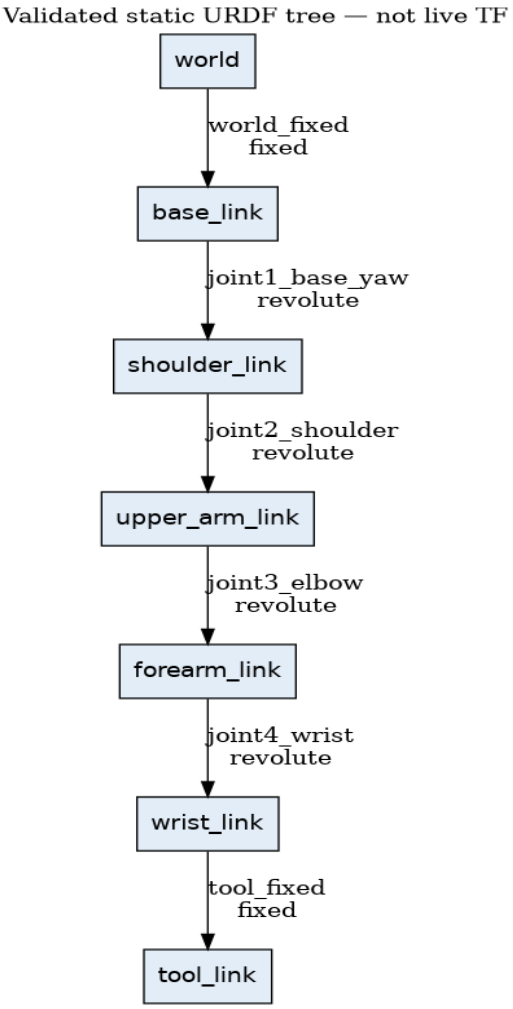
The launch file reads the installed URDF from the package share directory, gives robot_description to robot_state_publisher and joint_state_publisher_gui, and starts RViz with arm.rviz. joint_state_publisher_gui publishes movable joint coordinates on /joint_states. robot_state_publisher turns those values and the fixed URDF relations into TF transforms.

In RViz use Fixed Frame=world. Enable RobotModel, Grid and TF. RobotModel reads the transient-local /robot_description topic, with reliable QoS. The config starts an orbit camera focused near the arm. If a local RViz version does not restore a display property, configure that display manually and save the settings.

Runtime acceptance checklist: RobotModel status OK; all six physical links visible; four GUI sliders; changing joint1 rotates the full arm about Z; changing joint2/3/4 produces local Y-axis motion of the downstream chain; no detached segments or missing transforms. Capture home and bent screenshots with statuses visible. No real RViz run or screenshot was possible in this environment.

Troubleshooting: source both ROS and the workspace; confirm package discovery with ros2 pkg prefix arm_description; check /joint_states and /robot_description; choose world rather than an unknown fixed frame; inspect terminal output for dependency or display errors. Static syntax validation is not equivalent to successful launch.

6. TF tree and runtime verification (Q6)



The graph is generated from parsed parent-child relations and is a verified static expectation, not a captured live TF tree. world_fixed and tool_fixed should be published as fixed transforms on /tf_static. The four revolute transforms should appear on /tf when valid /joint_states messages arrive. A GUI pose is only reflected in RViz after that publishing chain is functioning.

```
# Run in another terminal with ROS + workspace sourced:
ros2 topic echo /joint_states --once
ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo world tool_link
```

At home, tf2_echo should report tool-frame translation approximately [0, 0, 0.87] m and identity orientation. view_frames should show one connected chain with no duplicate parents. Capture the actual generated frames PDF and tool transform output; filenames can vary by ROS distribution. Runtime timestamps, topic QoS and frame availability cannot be verified by the static graph alone.

7. Results, limitations and conclusion

Check	Actual result
URDF parser	PASS: urdf_parser_py 0.0.4 parsed the complete model
Topology	PASS: 7 links, 6 joints, 4 revolute DOF; root world
Geometry and inertia	PASS: visual/collision for 6 physical links; valid inertias
Joint configuration	PASS: normalized axes and ordered positive-capacity limits
Forward kinematics	PASS: 4 analytic target poses and invalid-angle rejection
Launch/config files	PASS: Python syntax and YAML structure only
ROS2 build and check_urdf	NOT RUN: ROS2/urdfdom command-line tools absent
RViz and live TF	NOT RUN: ROS2 runtime unavailable

The Python URDF parser was installed and actually used, rather than relying only on XML syntax. NumPy checks independently validated inertias and composed transforms. Parser success and mathematical checks support the structural correctness of the supplied description, but do not establish RViz compatibility or a running TF pipeline. The C++ check_urdf tool was not executed.

The model and package address the design and file-implementation tasks. Q5 and Q6 include exact runtime procedures but remain unverified until run in ROS 2. There is no dynamic simulation, ros2_control interface, path planner, real robot driver, payload rating or self-collision proof. Effort, velocity and mass values are explicit educational assumptions.

Conclusion: A complete serial 4-DOF arm description with visual/collision/inertial geometry, bounded joints, launch support and RViz configuration has been prepared. Static model and kinematic checks passed. Real RViz loading and live TF demonstration are still required to satisfy every assignment deliverable; no fabricated screenshots are included.

8. Submission files and references

Required item	Location / completion
Project report PDF	report.pdf (this report)
URDF / Xacro	arm_description/urdf/industrial_arm.urdf
Launch file	arm_description/launch/display.launch.py
RViz configuration	arm_description/rviz/arm.rviz
Runtime screenshots	Pending; evidence/SCREENSHOTS_REQUIRED.md
Validation evidence	evidence/validation_results.txt and fk_results.json
Source package	arm_description/package.xml and CMakeLists.txt

Before submission: build and launch the package on ROS 2 Humble; capture genuine RViz and live TF evidence; update this report with actual results. Upload the full folder ROS2_URDF_4DOF_Arm_Deep_Bisen to Google Drive and submit its shared link, as required in the assignment. A PDF alone does not include all requested source files and runtime screenshots.

References:

1. Supplied ROS Mini Project specification: ROS 2 URDF of 4-DOF Industrial Arm.
2. ROS 2 Humble documentation: https://docs.ros.org/en/humble/
3. URDF documentation and tutorials:
https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html
4. robot_state_publisher: https://docs.ros.org/en/humble/p/robot_state_publisher/
5. tf2 tools and tutorials: https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html
6. URDF Python parser: https://github.com/ros/urdf_parser_py

All model dimensions and masses were selected for this educational design. The visuals are derived from supplied geometry and the TF graph from supplied joint relations. No physical robot measurements or ROS runtime observations are represented as completed results.